

Sicurezza

CdL in Informatica – L31

Lezione L9

Operating System Security

Prof. Emiliano Casalicchio

Department of Computer Science

Sapienza University of Rome



SAPIENZA
UNIVERSITÀ DI ROMA



OS and Security

Is the **first line of defense**

- protect one user from another
- ensure that critical areas of memory or storage are not overwritten by unauthorized processes
- perform identification and authentication of people and remote operations
- ensures fair sharing of critical hardware resources

Is the **target** for attack

- successfully compromising the OS is complete control over the machine and all its components.



OS functions that involve CS

Enforced sharing.

- Resources should be made available to users as appropriate.
- Sharing brings about the **need to guarantee integrity and consistency**

Interprocess communication and synchronization

- OS acts as a bridge between processes, responding to process requests for asynchronous communication with other processes or synchronization.
- Interprocess communication is **mediated by access control tables**

Protection of critical operating system data

- OS must maintain data by which it can enforce security
- If these data are not protected against unauthorized access (read, modify, and delete), the operating system cannot do the enforcement



OS functions that involve CS (cont'd)

Guaranteed fair service.

- Need to avoid user/process starvation (to guarantee availability).
- Hardware clocks combine with scheduling disciplines to provide **fairness**. Hardware facilities and data tables combine to provide **control**.

Interface to hardware, File and I/O device

- Mediate and control the access to the hw devices, I/O devices, File system (**access control**)

User Authentication

Memory protection

- To guarantee user/system space is not violated (**CIA**)



Operating System Design to Protect Objects

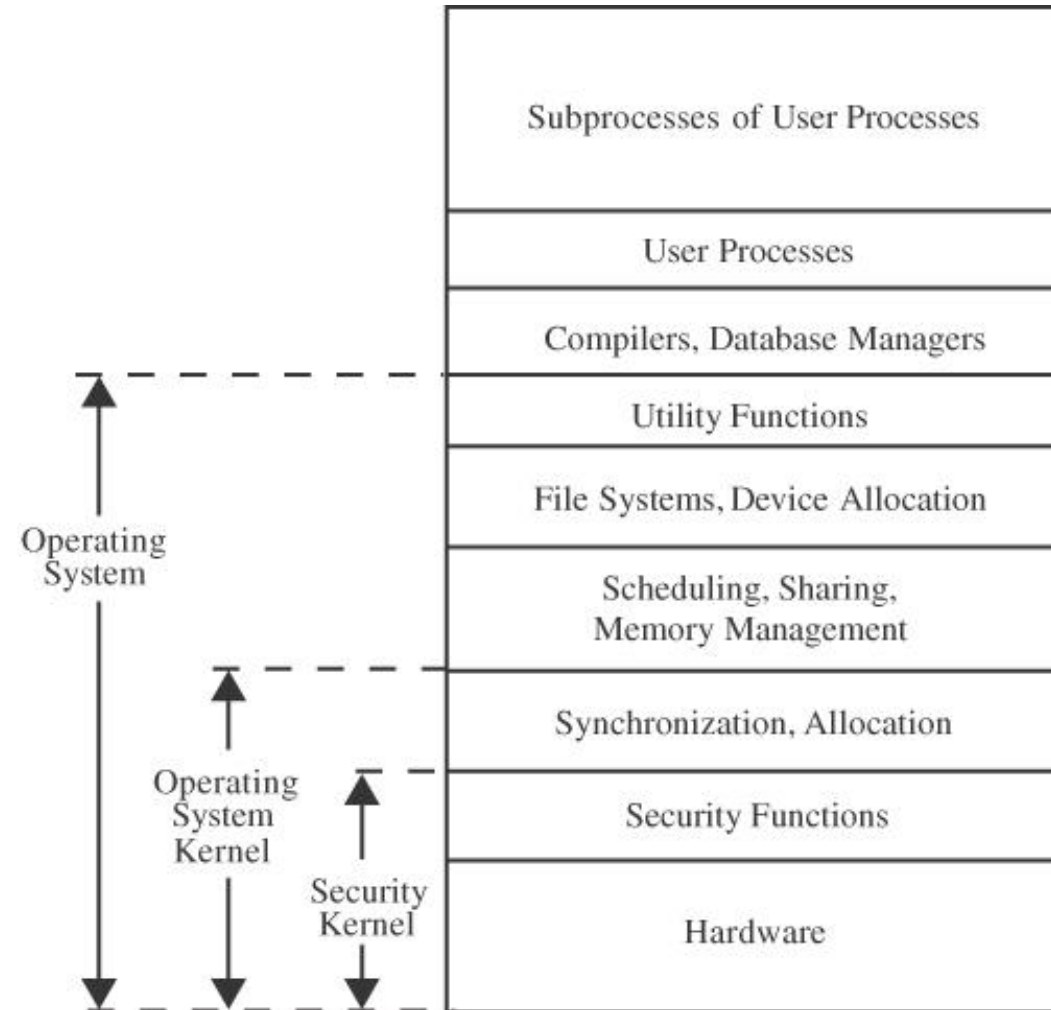
Object to be protected

- memory
- sharable I/O devices, such as disks
- serially reusable I/O devices, such as printers and tape drives
- sharable programs and subprocedures
- networks
- sharable data

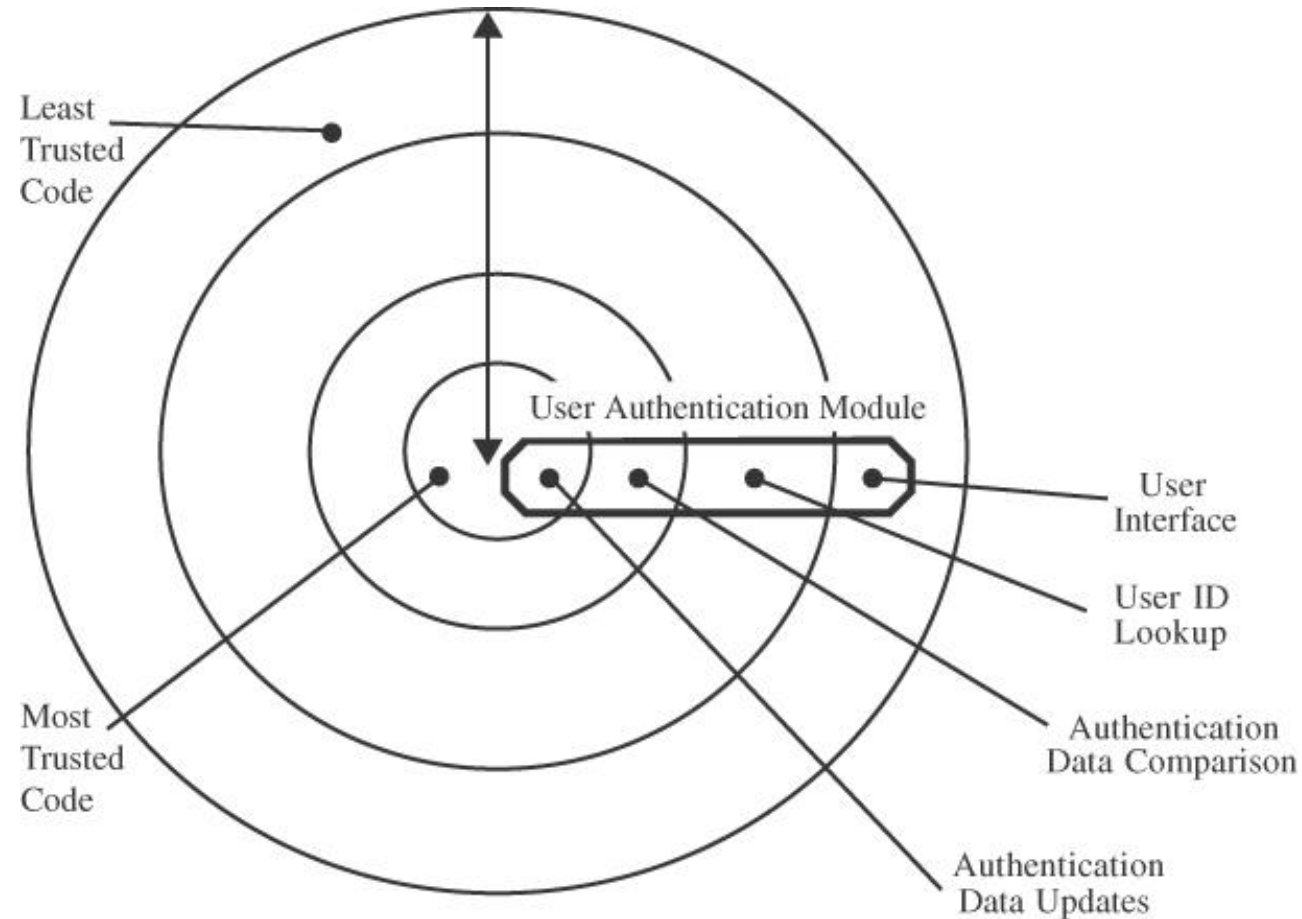


Layered view of OS

- security kernel to enforce security
- operating system kernel to allocate primitive resources such as time or access to hardware devices
- other operating system functions, to implement the user's interface to hardware



Onion-ring view and multi layer implementation of a security function



Multi component nature is a potential breach for security

From a security standpoint OS modules come from different sources, not all trustworthy, and must all integrate successfully

The OS must protect itself

- from errant or malicious user programs
- from harm from incorporated modules, drivers, and add-ons, and with limited knowledge of which ones to trust and for what capabilities



The HW enforced protection example

If the OS is developed for a specific HW, protection can be supported/implementd in HW

- VAX family used a hardware design that implemented four distinct protection levels
 - This structure put essentially three distinct walls around the most critical functions, including those that implemented security
- A BiiN operating system could support 64000 levels of separation/protection enforced by HW

If the OS is developed to run on generic HW the HW support/enforcement can not be leveraged



SECUK

~~OS tools to implement utility
functions~~



Already mentioned!!!

- ACL, privilege list and capabilities
 - OS implement both the underlying tables supporting **access control** and the mechanisms that check for acceptable uses
- Events logging
 - OS logs many events, such log are used for Audit
 - Audit logs show what happened in an incident; analysis of logs can guide prevention of future successful strikes



Virtualization

- Allow the OS to presents each user with just the resources that class of user should see
 - the virtual machine (VM) abstraction
- A **hypervisor**, or **virtual machine monitor**
 - Control and Filters all the request from a VM.
 - Execute in a controlled way privileged instructions and control resource allocation (e.g. memory access)



Sandbox

- An environment from which a process can have only limited, controlled impact on outside resources
- The original design of the Java system was based on the sandbox concept
 - Trusted code executed with access to the environment
 - Untrusted code, e.g. downloaded from the internet (applet) executed in a sandbox



Sandbox: how it works

- The Java run-time environment is a kind of virtual machine that presents untrusted applets with an unescapable bounded subset of system resources.
- Java compilers and a tool called a bytecode verifier ensure that the system executes only well-formed Java commands
- A class loader utility is part of the virtual machine monitor to constrain untrusted applets to the safe sandbox space
- Finally, the Java Virtual Machine serves as a reference monitor to mediate all access requests.



Honeypot

- Usually employed in a network
 - shows a limited (safe) set of resources for the attacker;
 - Allow security people to monitor the attacker's activities in real time to learn more about the attacker's objectives, tools, techniques, and weaknesses;
 - the knowledge acquired is used to defend systems effectively.
- Same approach can be used in OS, running a honeypot VM
 - Allow also to engage with the opponents



Separation and Sharing



Separation

Separation occurs by space, time, access control, or cryptography

- Physical separation, by which different processes use different physical objects, such as separate printers for output requiring different levels of security
- Temporal separation, by which processes having different security requirements are executed at different times
- Logical separation, by which users operate under the illusion that no other processes exist, as when an operating system constrains a program's accesses so that the program cannot access objects outside its permitted domain
- Cryptographic separation, by which processes conceal their data and computations in such a way that they are unintelligible to outside processes



Sharing

- Separation is only half the medal
- We want to separate one user from another user's objects, but we also want to be able to provide sharing for some of those objects
- An OS can support separation and sharing in several ways, offering protection at any of several levels.



Sharing and protection

Do not protect

- appropriate when sensitive procedures are being run at separate times.

Isolate.

- different processes running concurrently are unaware of the presence of each other. Each process has its own address space, files, and other objects.
- The OS must confine each process somehow so that the objects of the other processes are completely concealed.

Share all or share nothing.

- the owner of an object declares it to be public or private.
- a public object is available to all users, whereas a private object is available only to its owner.



Sharing and protection (cont'd)

Share but limit access.

- Access control

Limit use of an object.

- This form of protection limits not just the access to an object but the use made of that object after it has been accessed.
- For example, a user may be allowed to view a sensitive document but not to print a copy of it

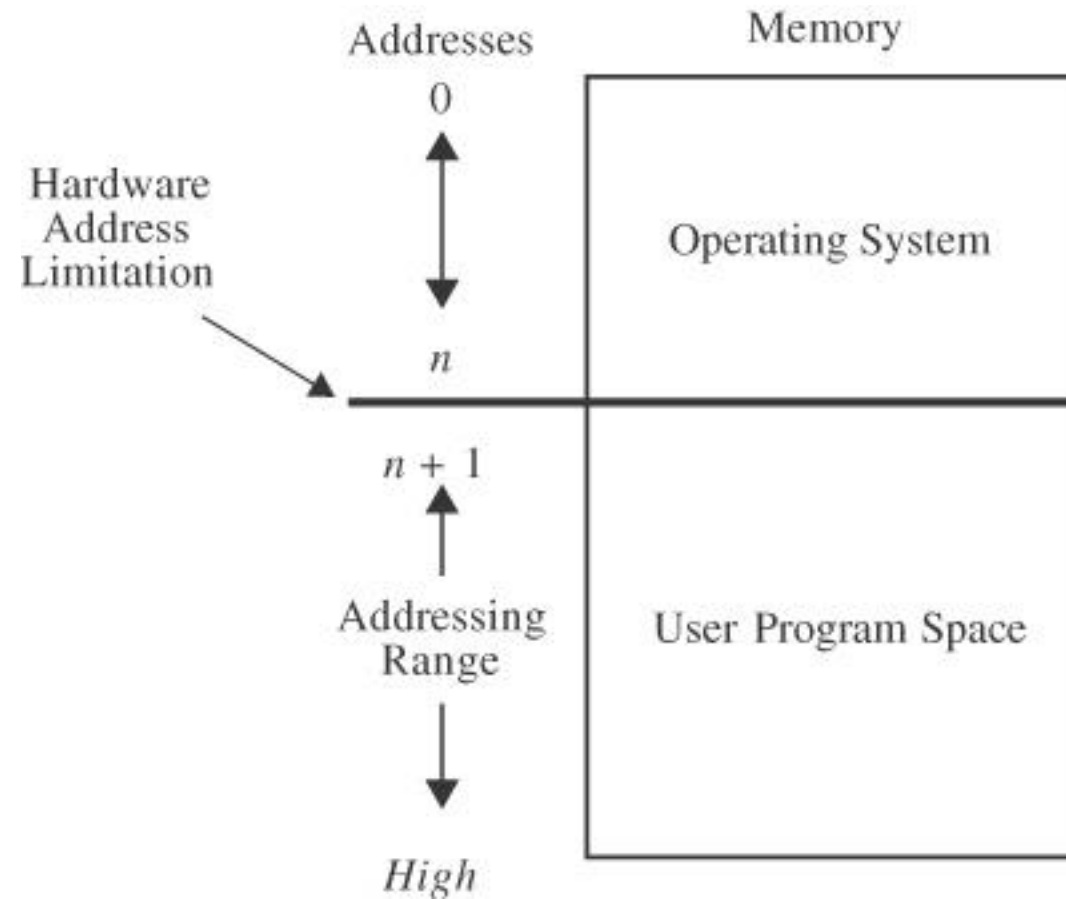


HW protection of Memory



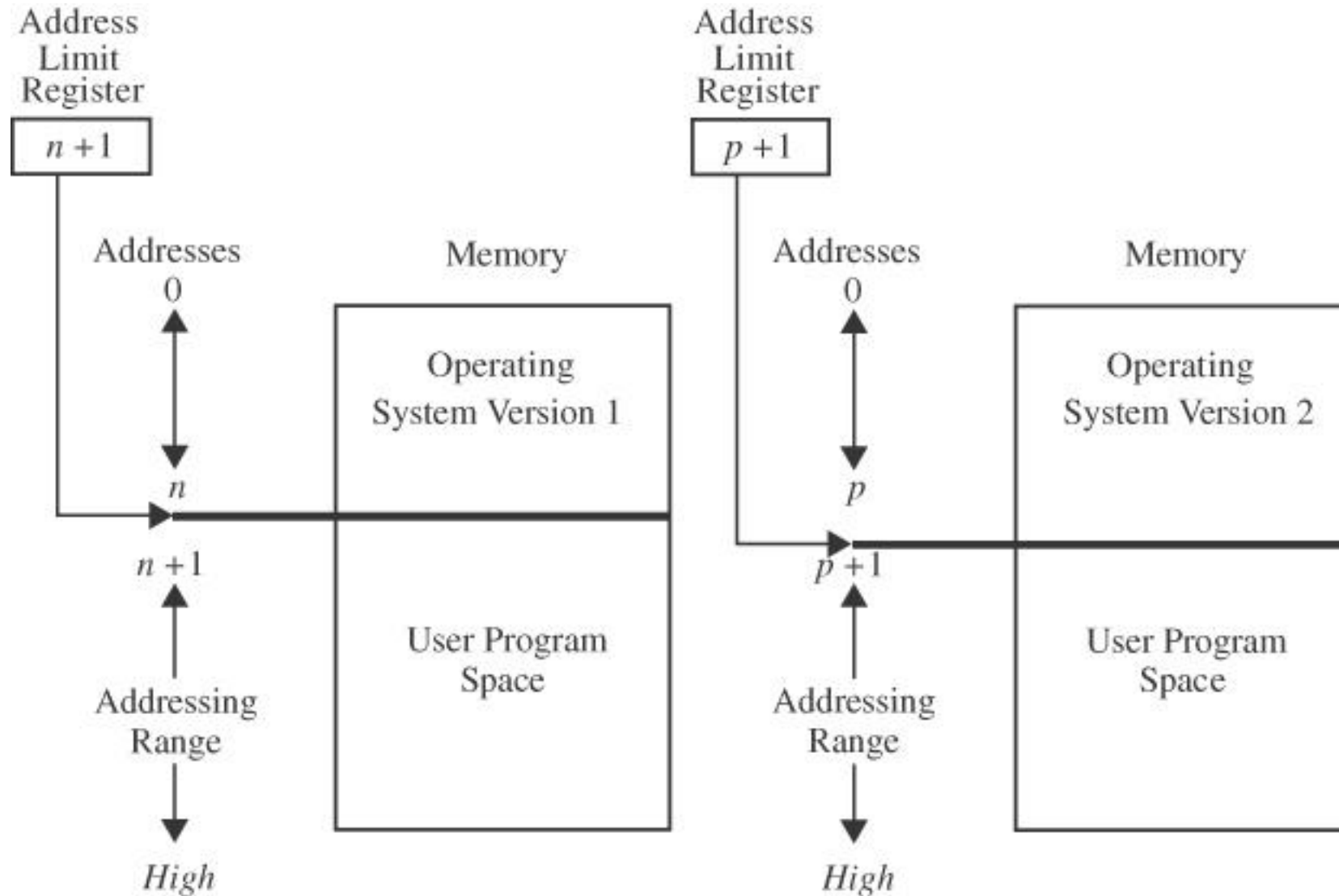
Fence

- Static

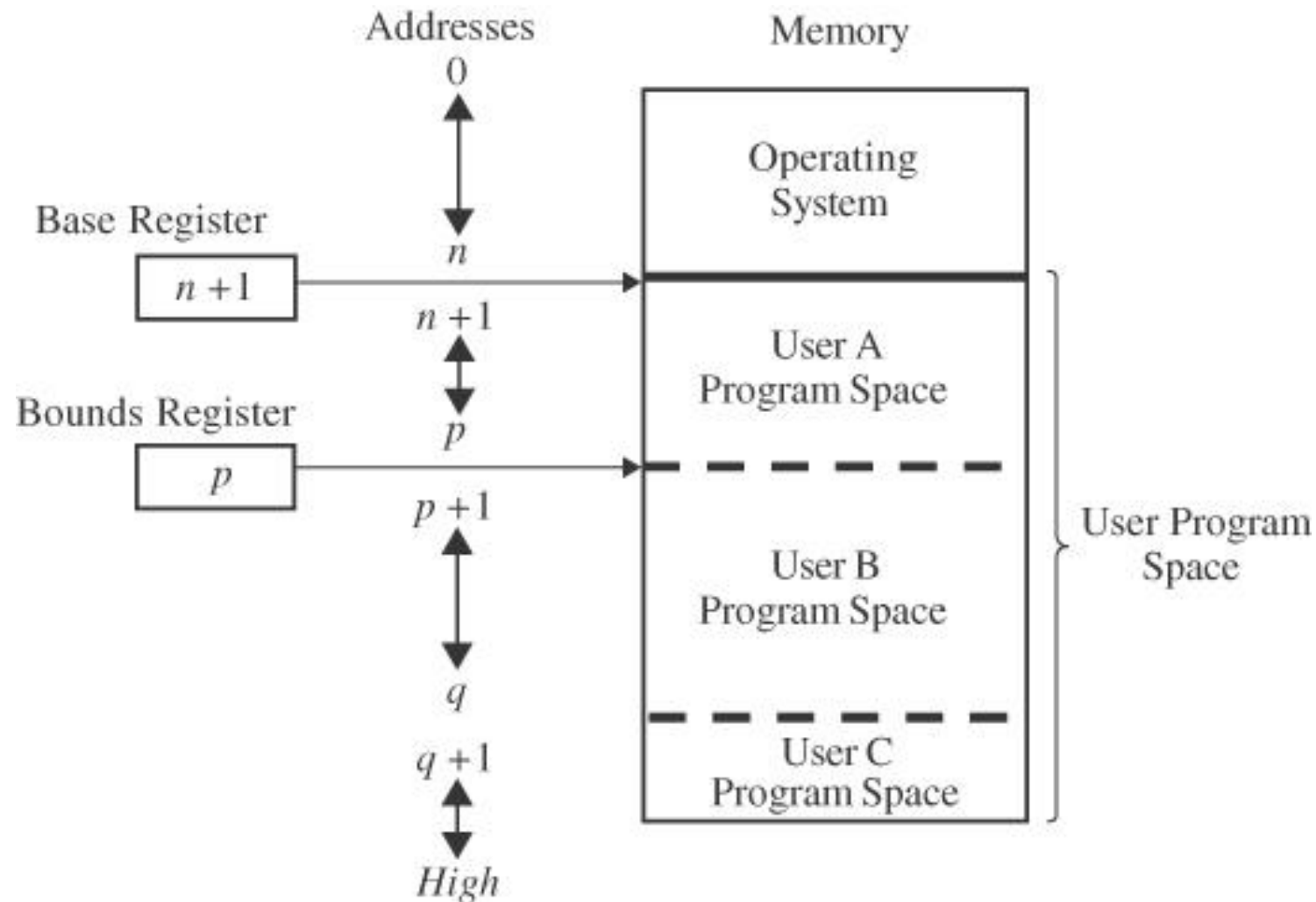


Fence Register

- Dynamic

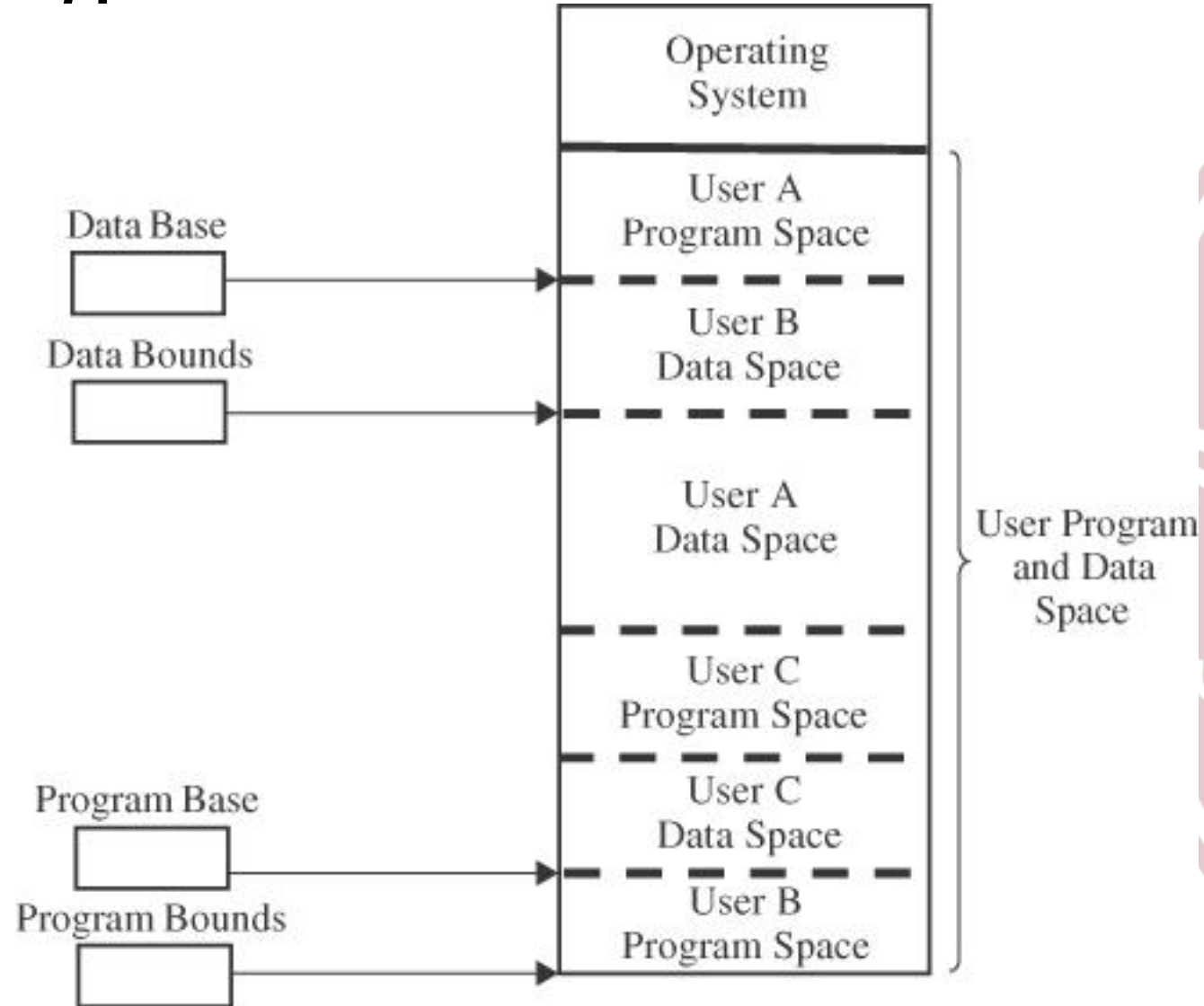


Base/bound registers



Base/bound registers

- For Data and Code

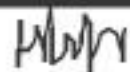
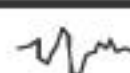
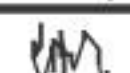
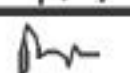
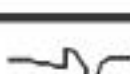


casali@ccr.it



Tagged Architecture

- Control each memory location

Tag	Memory Word
R	0001
RW	0137
R	0099
X	
X	
X	
X	
X	
X	
R	4091
RW	0002

Code: R = Read-only RW = Read/Write
X = Execute-only

casalicchio@di.uniroma1.it

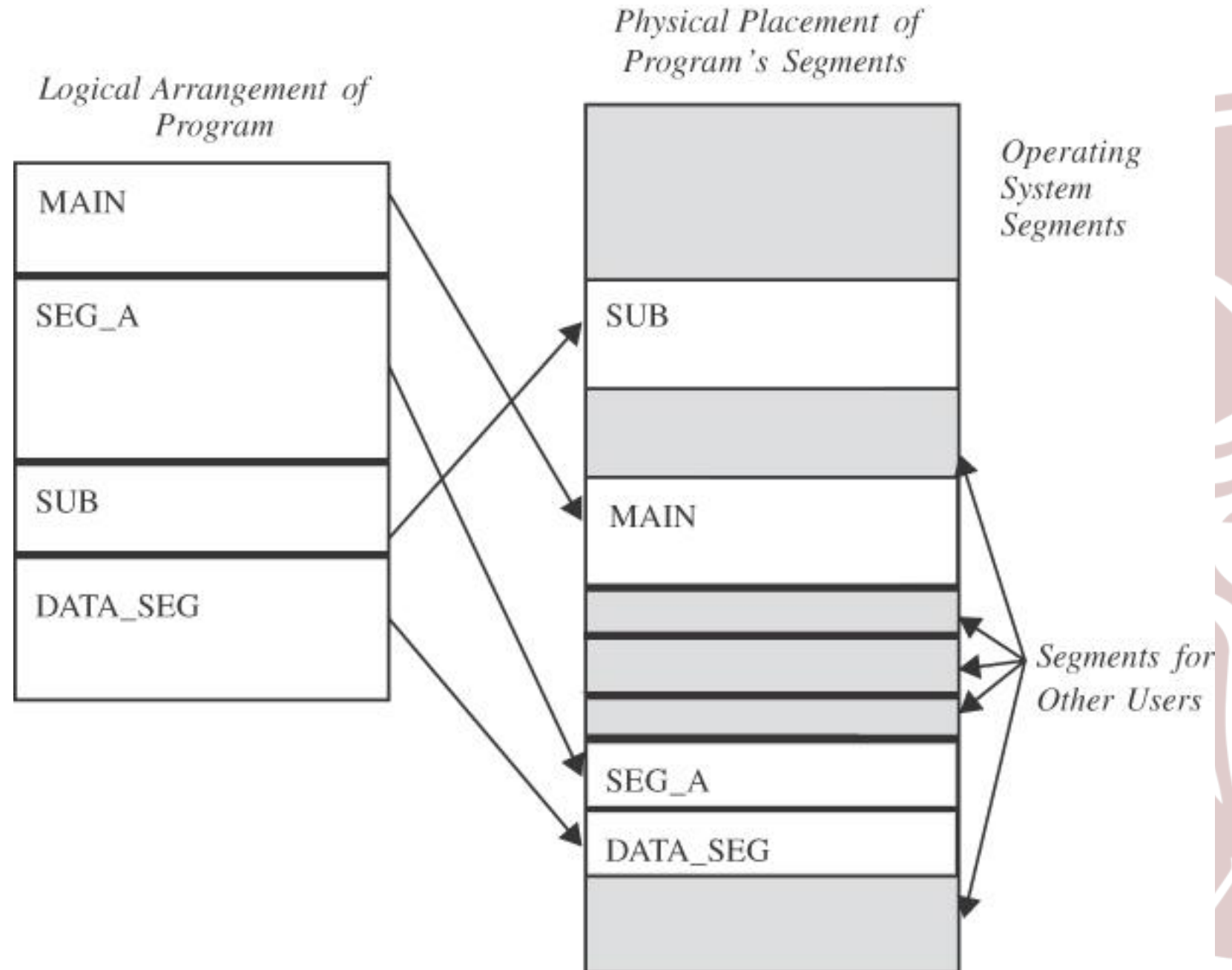


Virtual Memory



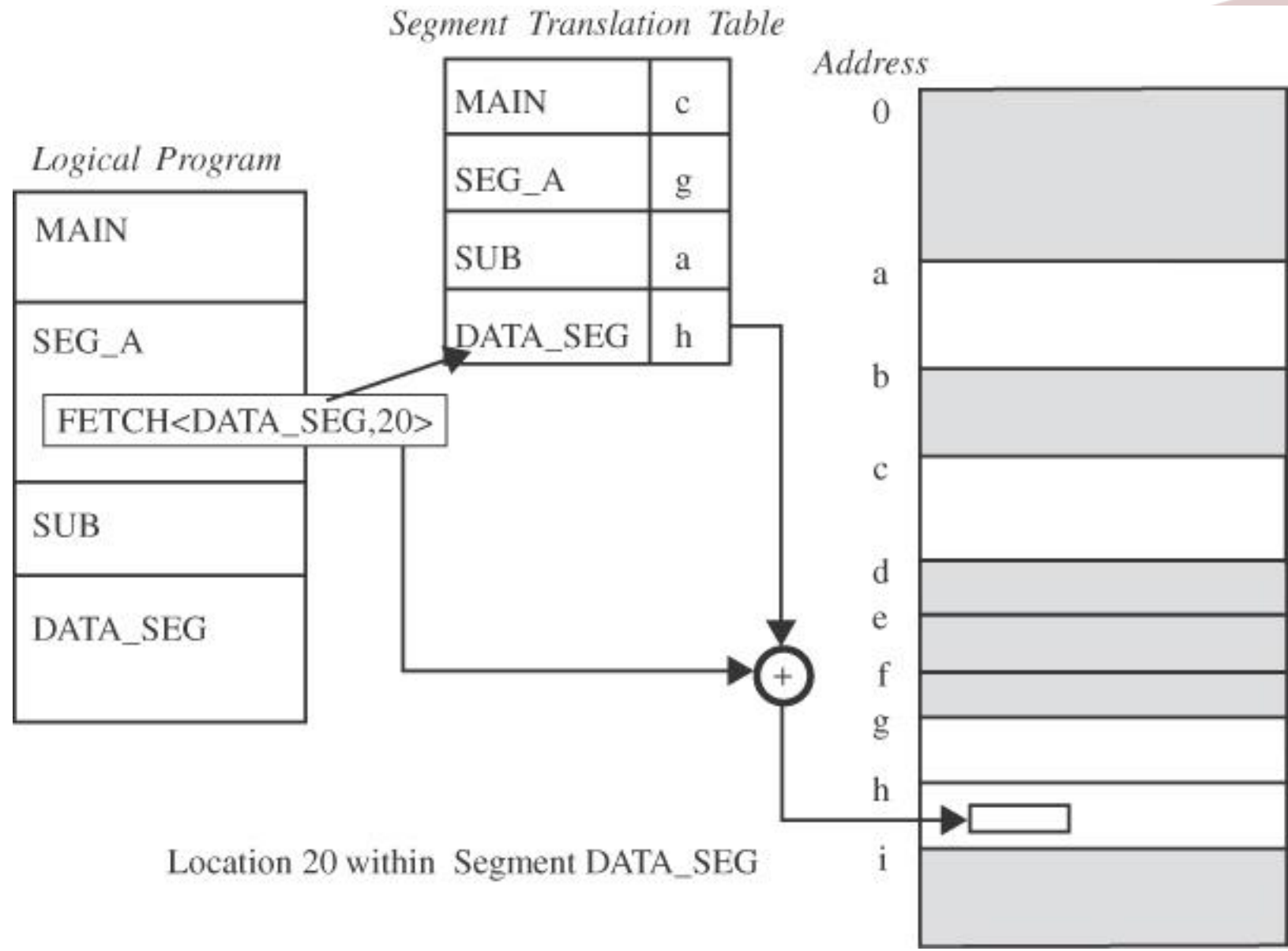
Segmentation

- allows a program to be divided into many pieces having different access rights
- Implement infinite base bound registers



Segmentation

- Implemented through address translation
- Addresses are hidden



Hiding of addresses advantages

- The operating system can place any segment at any location or move any segment to any location, even after the program begins to execute
- A segment can be removed from main memory (and stored on an auxiliary device) if it is not being used currently
- Every address reference passes through the OS,
 - **there is an opportunity to check for protection and implement access control**



Security benefits of segmentation

- Each address reference is checked—strictly within the size of the segment—for protection
- Many different classes of data items can be assigned different levels of protection
- Two or more users can share access to a segment, with potentially different access rights
- A user cannot generate an address or access to an unpermitted segment



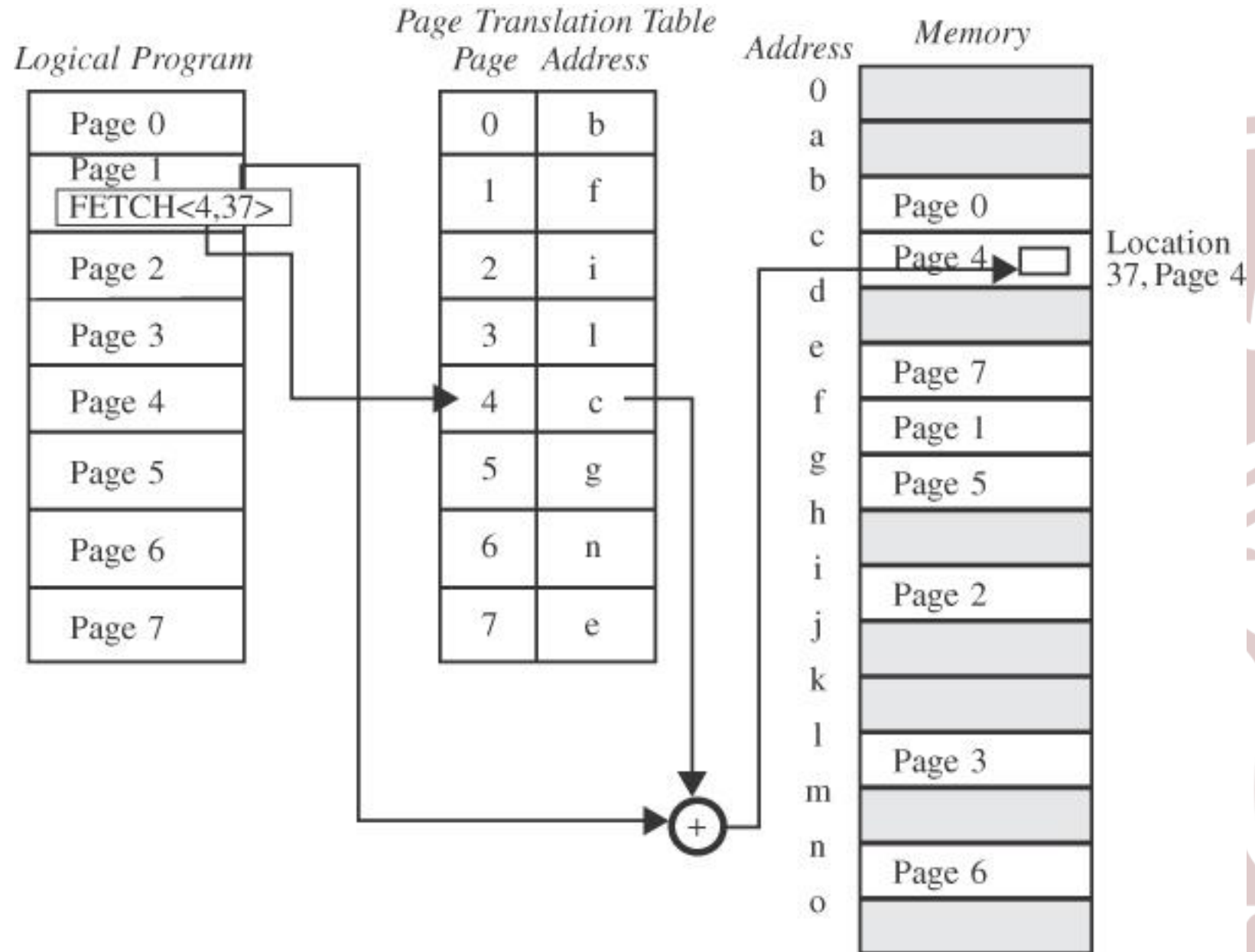
Issues with segmentation

- Segment size
- secure implementation of segmentation requires the checking of a generated address to verify that it is not beyond the current end of the segment referenced

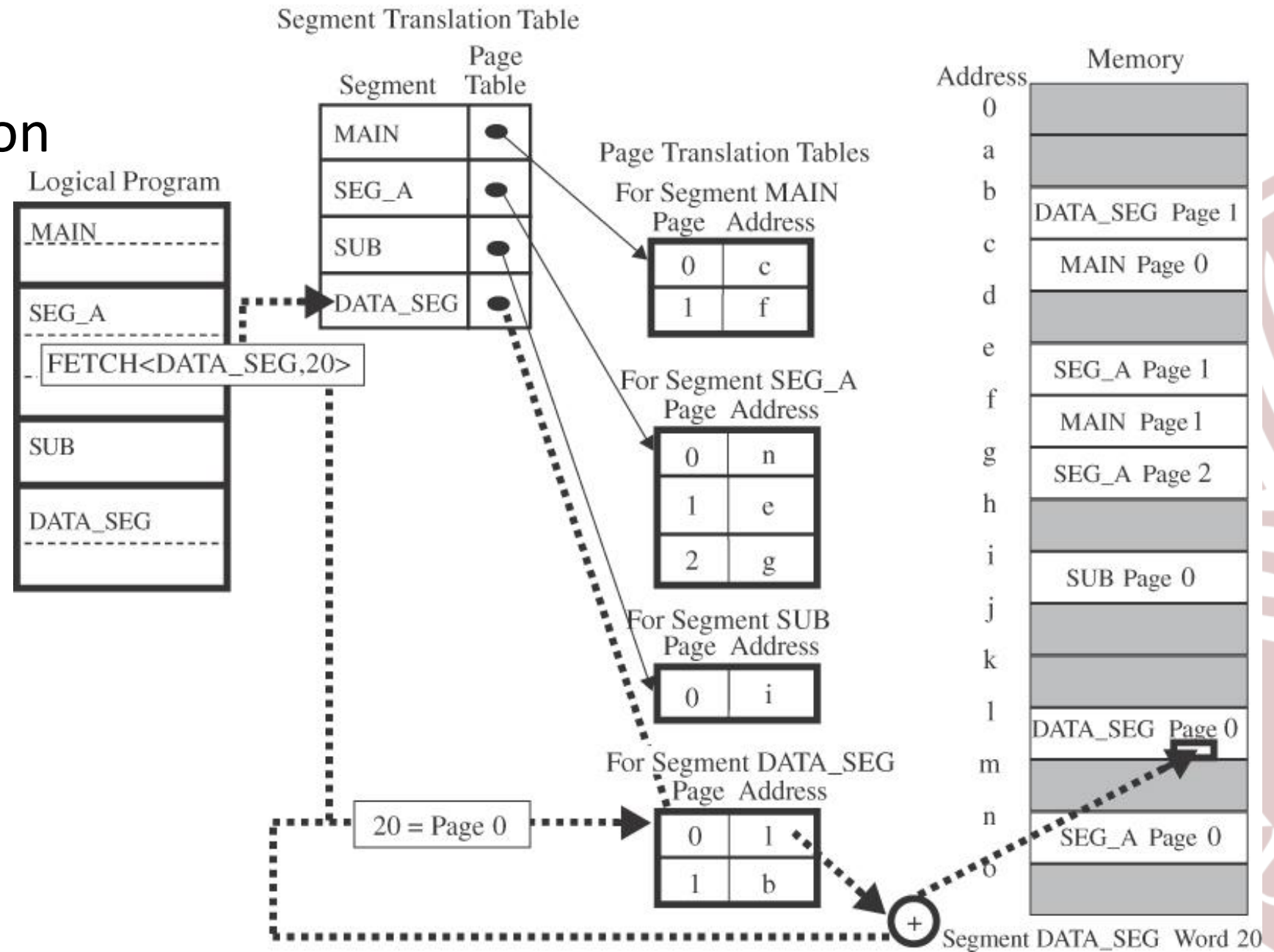


Paging

- The program is divided into equal-sized pieces called **pages**, and memory is divided into equal-sized units called **page frames**
- **Paging allows the security advantages of segmentation with more efficient memory management**



Paged Segmentation



Security in the Design of Operating Systems



Security in the Design of OS

- Simplicity of Design
- Layered Design, Layered trust
 - Achieve encapsulation, damage control
- Kernelized Design, Security Kernel
 - Coverage, Isolation, Unity, Modifiability, Compactness, Verifiability
- Reference Monitor (part of the security kernel)
 - Access control
- Correctness and Completeness
 - Security enforcement must be correct and complete
- Security Design principles
 - least privilege, economy of mechanism, open design, complete mediation, permission based, separation of privilege, least common mechanism, ease of use



Security in the Design of OS (Cont'd)

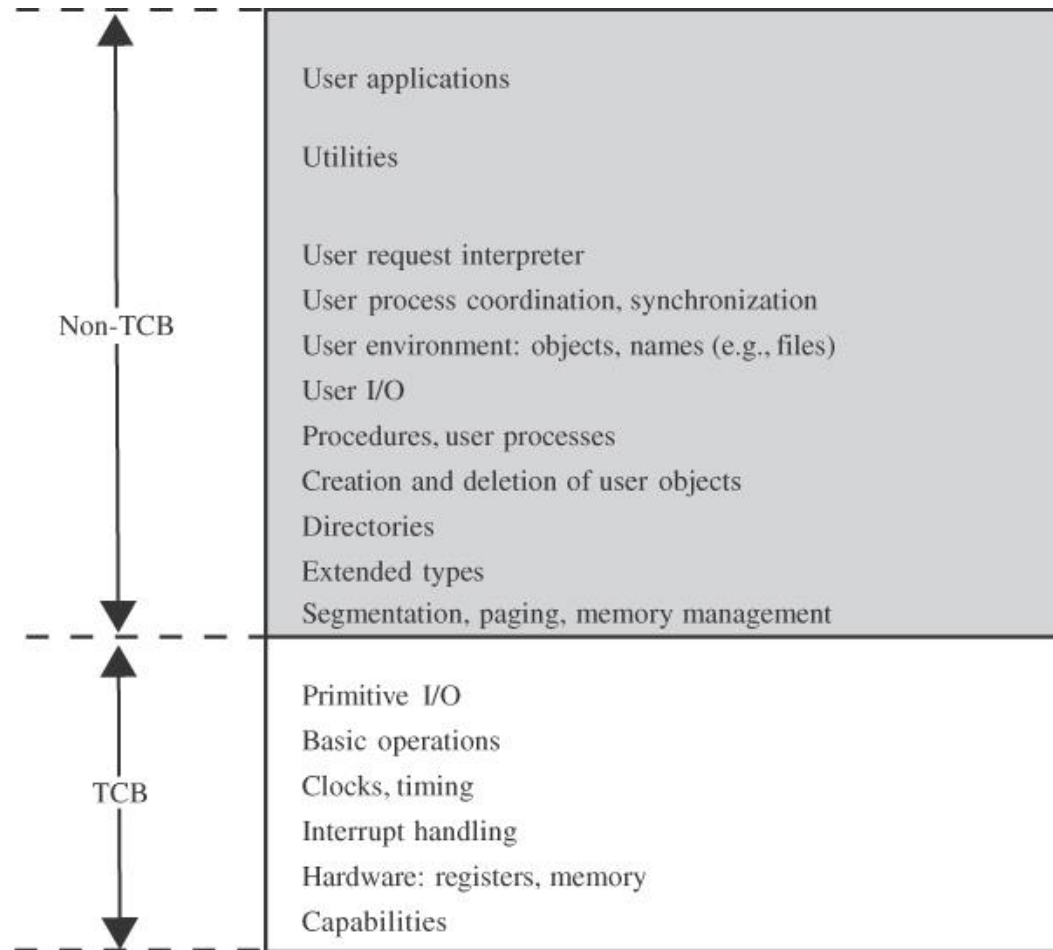
Trusted Systems

- Key feature
 - Functional correctness. The program does what it is supposed to, and it works correctly.
 - Enforcement of integrity. Even if presented erroneous commands or commands from unauthorized users, the program maintains the correctness of the data with which it has contact.
 - Limited privilege. The program is allowed to access secure data, but the access is minimized and neither the access rights nor the data are passed along to other untrusted programs or back to an untrusted caller.
 - Appropriate confidence level. The program has been examined and rated at a degree of trust appropriate for the kind of data and environment in which it is to be used
- Characteristics
 - a defined policy that details what security qualities it enforces
 - appropriate measures and mechanisms by which it can enforce that security adequately
 - independent scrutiny or evaluation to ensure that the mechanisms have been selected and implemented properly so that the security policy is in fact enforced.



Trusted Computing Base

- is the name we give to everything in the trusted operating system that is necessary to enforce the security policy
- TCB forms the fortress-like shell that protects whatever in the system needs protection
- TCB depends/built on
 - HW, processes, primitive files, protected memory, IPC



What TCB monitors to maintain the secrecy and integrity of each domain

- Process activation
- Execution domain switching
- Memory protection
- I/O operation



TCB implementation

- Collecting all security functions into the TCB may destroy the modularity of an existing operating system
- A unified TCB may be too large to be analyzed easily
- Creating a security kernel with security functions separated from the other OS functions
- Building firstly the security kernel and then the OS around. This make the security kernel the interface with the HW

